toolbar.py

```
from collections.abc import Callable
from PIL import Image, ImageTk
import tkinter as tk
from tools import *

class ToolBar(tk.Frame):
    def __init__(self, master, on_tool_change: Callable[[Tool], None],*args, **kwargs):
        super().__init__(master = master, *args, **kwargs)

        self.configure(
            background = "lightgray",
            width = 64,
            relief = "raised",
            border = 2
        )

        self.pack_propagate(False)

        self.callback = on_tool_change

        self.currentTool = "pencil"

        # 16 x 16 icons in a row scales to self.toolIconSize
        self.toolIconSize = 16
        self.toolCount = 16
        self.toolIconSheet = Image.open("assets/tools.png").resize((
                                        self.toolIconSize * self.toolCount,
                                        self.toolIconSize),
                                        Image.NEAREST) # type: ignore

        self.toolIcons = [] # Made so that images aren't garbage collected

        # Names in order of image
        self.tools = {
            "freeform-selection": Tool(),
            "rectangular-selection": Tool(),
            "eraser": Eraser(),
            "paint-bucket": Bucket(),
            "eye-dropper": Eyedropper(),
            "zoom": Tool(),
            "pencil": Pencil(),
            "brush": Tool(),
            "spray-can": Tool(),
            "type": Tool(),
            "line": Line(),
            "curve": Tool(),
```

```
            "rectangle-shape": Rectangle(),
            "custom-shape": CustomShape(),
            "circle-shape": Ellipse(),
            "beveled-rectangle-shape": Tool()
        }

        self.wrapper = tk.Frame(self)

        i = 0 # Icon itterator

        for x in range(2):
            for y in range(8):
                frame = tk.Frame(
                    master = self.wrapper,
                    relief = "raised",
                    border = 2,
                    name = list(self.tools)[i]
                )

                self.toolIcons.append(
                    ImageTk.PhotoImage(
                        self.toolIconSheet.crop((
                            i * self.toolIconSize,
                            0,
                            i * self.toolIconSize + self.toolIconSize,
                            self.toolIconSize
                        ))
                    )
                )

                # Label for image
                tk.Label(
                    frame,
                    image = self.toolIcons[-1]).pack()

                frame.grid(
                    column = x,
                    row = y
                )

                # Bind click to label and not frame
                for widget in frame.winfo_children():
                    widget.bind("<Button-1>", lambda _, name = list(self.tools)[i]:
self.setTool(name))

                i -=- 1 # Lol


        self.wrapper.pack(anchor = "n", expand = True, pady = 5)
```

```
95
96          # Set relief of current tool initially
97          self.setTool("pencil")
98
99      def setTool(self, toolName: str):
100
101         # Reset style of current
102         self.wrapper.nametowidget(
103             self.currentTool
104         ).configure(
105             relief = "raised"
106         )
107
108         # Set new tool and style
109         self.currentTool = toolName
110
111         self.wrapper.nametowidget(
112             toolName
113         ).configure(
114             relief = "sunken"
115         )
116
117         # Paint callback
118         self.callback(self.tools[self.currentTool])
119
120     def getTool(self) -> Tool:
121         return self.tools[self.currentTool]
122
```